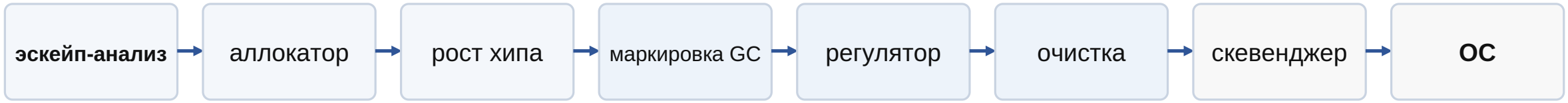
Доп. лекция 3. Внутреннее устройство GC и аллокатора памяти

mcache → mspan → mcentral → mheap · классы размеров · Green Tea · барьер записи · регулятор GC помощь GC · очистка спанов · скевенджер

Углубленная лекция о рантайме. Версия реализации: Go 1.27.x.

Карта внутренних механизмов



Что делает рантайм после решения компилятора: «нужна аллокация в хипе»?

Аллокатор, GC, планировщик и виртуальная память ОС работают как единая система.

Сразу разделяем модель и реализацию

Устойчивая инженерная модель

- быстрый путь на каждом P
- выделение по классам размеров
- конкурентный трассирующий GC
- барьеры записи
- регулирование работы GC
- помощь GC со стороны программы
- очистка + возврат памяти ОС

Детали текущей реализации

- `mcache`, `mspan`, `mcentral`, `mheap`
- конкретные битовые карты и поля
- внутренние очереди Green Tea
- константы регулятора GC
- правила применения `madvise`
- функции Go 1.27 под конкретные размеры

Исходники рантайма ≠ спецификация языка.

Почему «рантайм выделил память» — слишком простая модель


```
func newUser() *User {
    u := User{ID: 42}
    return &u
}
```

компилятор: u уходит в хип

Кто найдет свободную память?

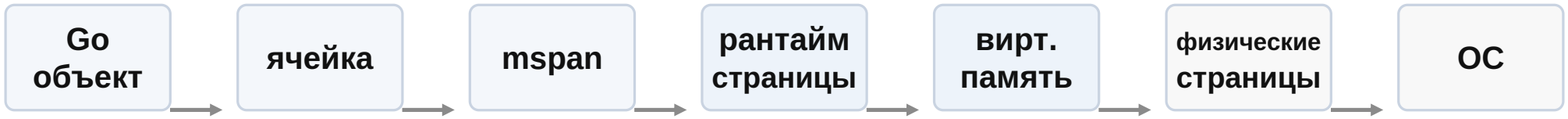
Нужен ли лок?

Будет ли системный вызов?

Кто затем вернет память?

Аллокация в хипе запускает цепочку рантайма, а не один вызов «дай память».

Память существует на разных уровнях



ОС не знает об объекте `User`. Для ядра это страницы виртуальной памяти процесса.

Почему аллокатор многоуровневый

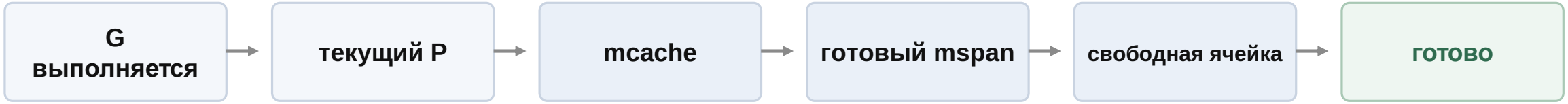
тысячи горутин → несколько P → локальное состояние аллокатора

частый путь: без глобального лока аллокатора

редкий медленный путь: общие структуры → аллокатор страниц → ОС

локальный быстрый путь + общий медленный

Быстрый путь небольшой аллокации



Без mcentral, mheap, mmap и глобального лока.

Поэтому небольшая аллокация на быстром пути может быть очень дешевой.

Почему кеш аллокатора привязан к P



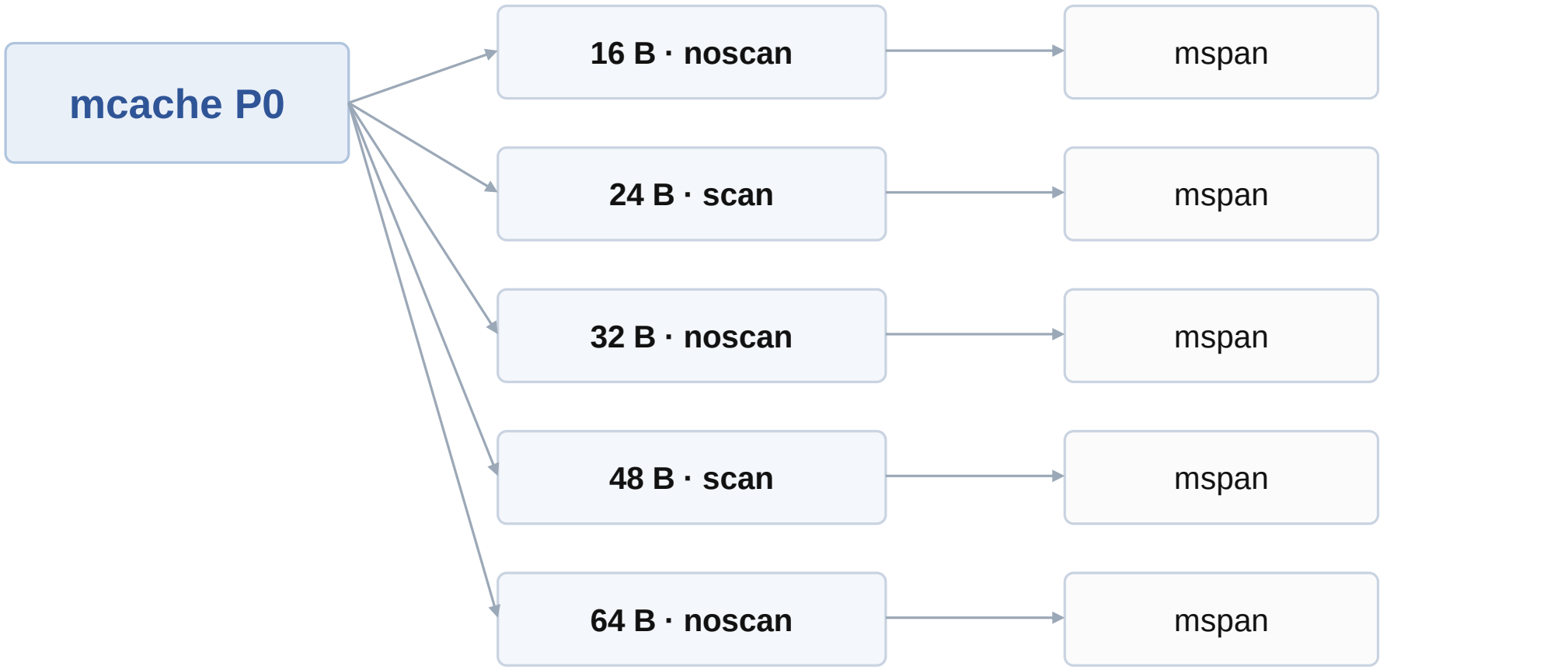
Не для каждой горутины

Не один общий кеш

≈ реальная параллельность

У P есть локальные данные планировщика и аллокатора: один принцип локальности.

mcache: набор готовых спанов, а не участок памяти



размер + наличие указателей → spanClass → готовый спан

Классы размеров: расходуем память ради быстрого пути


```
запрос = 37 Б
класс = 48 Б
ячейка = 48 Б

потери = 11 Б
```

Больше классов → **меньше потерь, выше сложность**

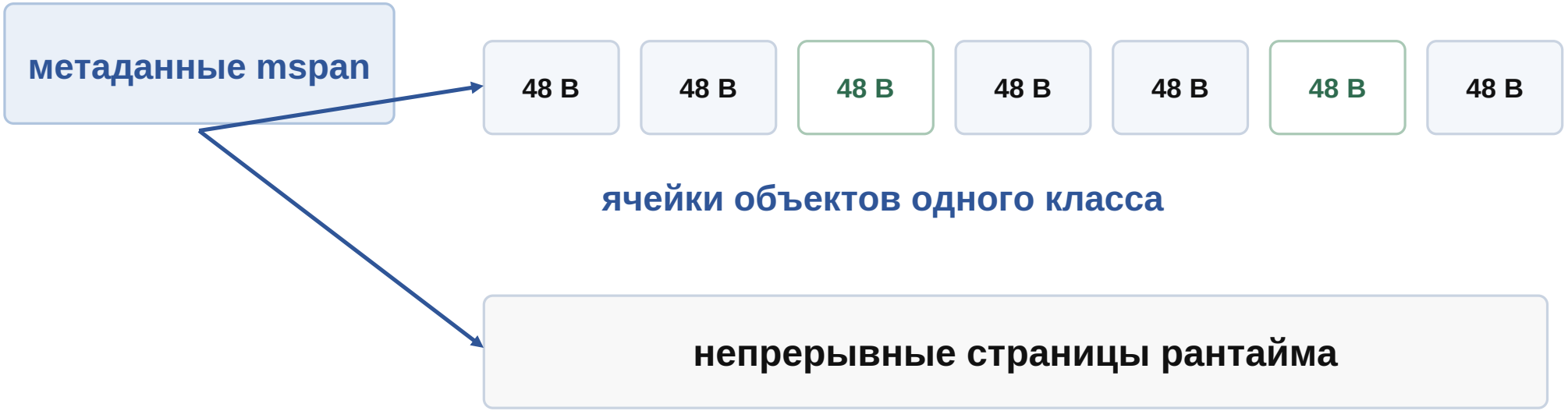
Меньше классов → **проще аллокатор, больше фрагментация**

Малые объекты: классы до 32 КиБ.

Таблица классов размеров — деталь реализации Go.

Источник: runtime sizeclasses.go

mspan: центральная единица аллокатора



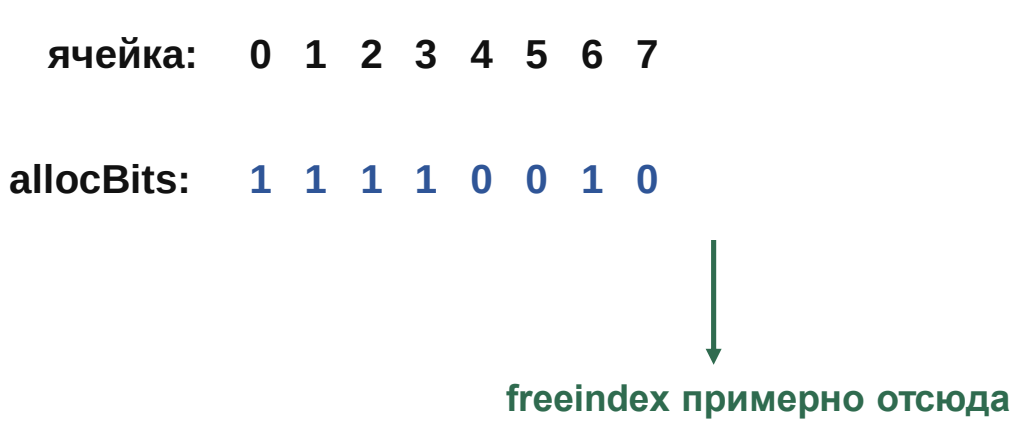
mspan ≠ один объект

один спан содержит множество ячеек

mspan в цифрах: ячейки, allocBits, freeindex

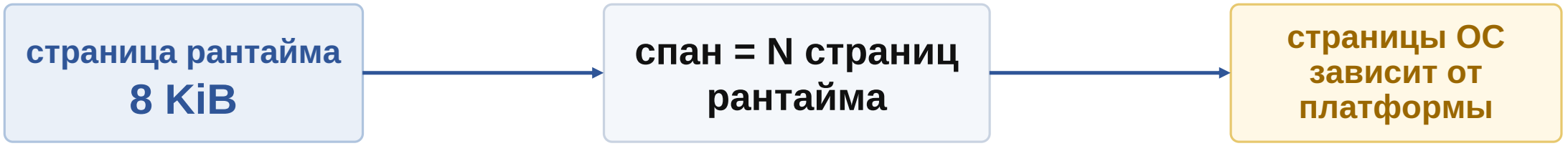

```
страница рантайма = 8192 Б
размер ячейки = 64 Б

  8192 / 64 = 128 ячеек
```



Аллокатор ищет свободную ячейку по битовой карте, без вызова ОС на каждый объект.

Страница рантайма ≠ физическая страница ОС



Здесь абстракция рантайма соприкасается с виртуальной памятью ОС.

8 КиБ — деталь рантайма, а не гарантия языка.

scan / noscan: одинаковый размер, разная цена GC

Объект без указателей

```
type A struct {
    X uint64
    Y uint64
}
```

Объект с указателями

```
type B struct {
    X *User
    Y uint64
}
```

noscan

scan

spanClass = класс размера + бит noscan

100 MB `[]byte` и 100 MB `[]*Node` — не одно и то же

[]byte · 100 MB

```
byte byte byte byte ...

GC: нет указателей на объекты хипа
```

[]*Node · 100 MB

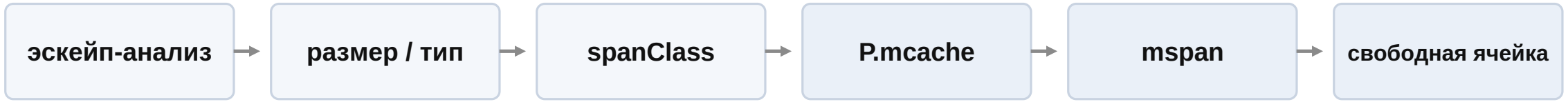
```
ptr ptr ptr ptr ...

GC: сканировать указатели
    и обходить граф объектов
```

тот же размер хипа ≠ та же работа GC

Практическая метрика: /gc/scan/heap:bytes.

Полный путь небольшой аллокации



ячейка есть → выдать

нет ячейки → mcentral → mheap → аллокатор страниц

ОС подключается, когда рантайму действительно нужна новая область памяти.

Обнуление и аллокации по размерам в Go 1.27

Зачем обнулять память

```
старая ячейка:
DE AD BE EF ...

новое значение:
0 0 0 0 ...
```

Что оптимизирует Go 1.27

- компилятор уже знает размер
- знает scan/noscan
- для части объектов <80 Б применяет специальные функции
- меньше общих проверок и косвенных вызовов

Граница компилятора и рантайма тоже оптимизируется.

Порог и набор функций зависят от версии Go.

Источник: go.dev/blog/size-specialized-allocations

Когда mspan заполнен: пополнение



Пополнение реже, чем выдача объектов внутри спана.

Одна дорогая операция окупается десятками или сотнями следующих аллокаций.

mcentral управляет спанами, а не объектами

mcentral[64 B · noscan]

неполные

span A | span B | span C

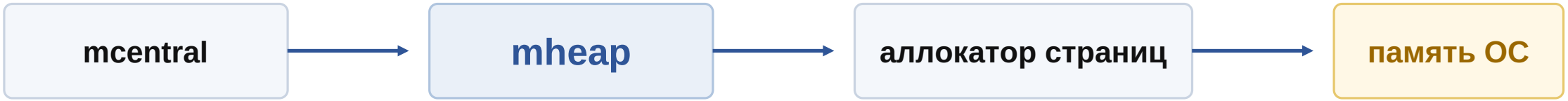
полные

span D | span E

mcache получает спан → **затем локально раздает ячейки**

Синхронизация вынесена с быстрого пути на путь пополнения.

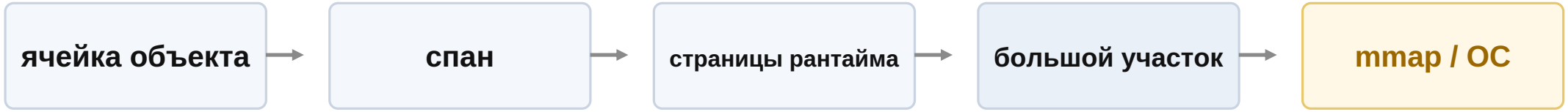
mheap и аллокатор страниц




```
карта страниц:
занято занято свободно свободно занято ...
              └────────┘
            новый спан
```

mheap работает со страницами, объекты раздаются через спаны уровнем выше.

Когда появляется ОС



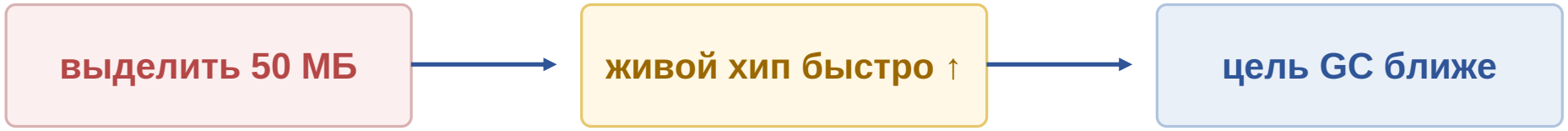
Аллокация 64 Б обычно не требует системного вызов.

Малые, большие и крошечные объекты: три пути

КРОШЕЧНЫЕ	МАЛЫЕ	БОЛЬШИЕ
<16 Б · без указателей несколько объектов в одном блоке	≤32 KiB классы размеров + mcache/mcentral	>32 KiB отдельный спан из страниц

Один аллокатор — разные пути для разных размеров.

Почему большие аллокации особенно заметны



фрагментация / RSS

возврат памяти ОС

момент запуска GC

Частые большие аллокации особенно заметны при пилообразной нагрузке.

Tiny-аллокатор: экономия на крошечных объектах


```
new(byte)
new(uint16)
new(uint32)
```

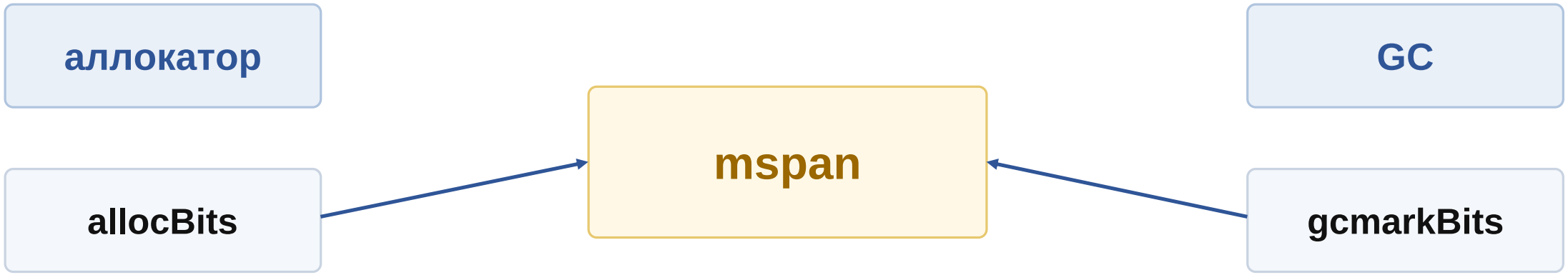
блок tiny: 16 Б

без указателей

1 B **2 B** **4 B** **...**

Цена: один живой вложенный объект может удерживать весь блок.

Где аллокатор встречается с GC



Объект умер ≠ ячейка сразу свободна. Сначала GC доказывает, что объект недостижим.

Архитектура Go GC

обход графа

точный

конкурентный

параллельный

маркировка и очистка

без поколений

без уплотнения

Это устройство сборщика мусора, а не набор ярлыков.

Что означают эти свойства

Механика

- обход графа → от корней по указателям
- точный → рантайм знает расположение указателей
- конкурентный → приложение работает при маркировке
- параллельный → несколько потоков работы GC

Следствие

- без поколений → нет деления на молодые и старые
- без уплотнения → живые объекты обычно не перемещаются ради уплотнения
- фрагментацию обрабатывают аллокатор и механизм управления страницами, не уплотнение

GC Go нельзя описывать моделью перемещающего сборщика с поколениями.

Изменение графа во время GC: зачем нужен барьер записи

GC просканировал A

A → B

программа пишет A → C

Сборщик и программа одновременно меняют граф объектов.

Для записи указателей нужен учет → барьер записи.

Гибридный барьер записи: учебная модель


```
до записи:
ячейка → старый

после записи:
ячейка → новый
```

GC не должен потерять важные указатели

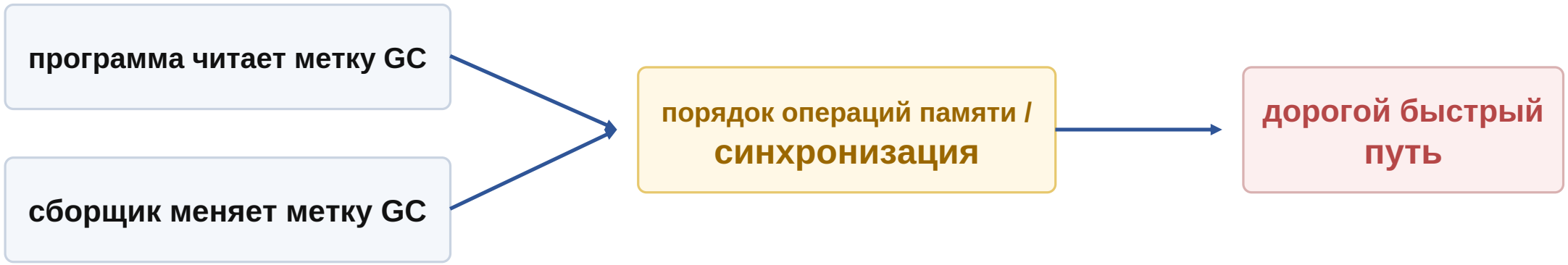
старый указатель

новый указатель

Барьер — дополнительная работа GC при записи во время маркировки.

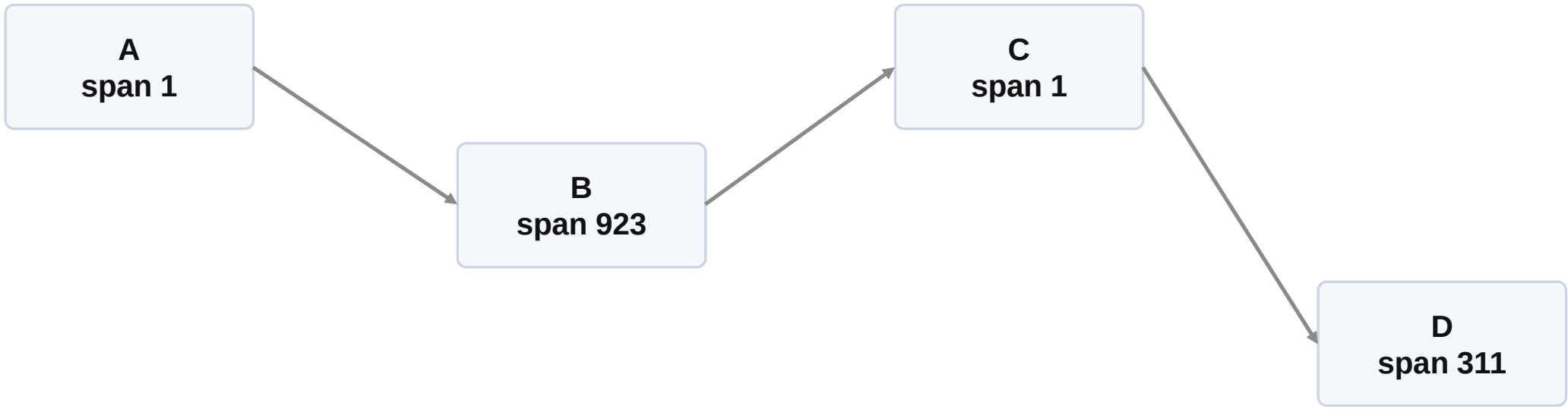
Инварианты Юасы/Дейкстры — детали рантайма. Суть: граф меняется при обходе.

Почему не проверять каждый раз «объект уже черный?»



Рантайм предпочитает лишнюю маркировку дорогой синхронизации при каждой записи.

Green Tea: проблема хаотичного обхода указателей



обход указателей → промахи кеша → ожидание DRAM

Маркировку часто ограничивает локальность памяти, а не вычисления.

Green Tea: группировка работы по спанам



повторное использование метаданных и строк кеша · меньше случайных переходов

С Go 1.26 Green Tea — по умолчанию

Изменилась реализация GC, а модель достижимости осталась прежней.

Источник: go.dev/blog/greenteagc

Маркировка ≠ сканирование: нужны два состояния

помечен = да

просканирован = нет

→ **объект уже живой, но исходящие указатели еще не обработаны**

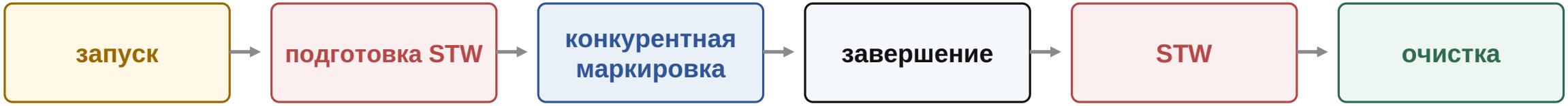
Работу маркировки можно группировать по спанам.

Один принцип повторяется во всем рантайме



Локальность сначала. Глобальная координация — позже.

Цикл GC: последовательность фаз



рабочие GC

скан. стеков

барьеры

помощь GC

STW остается для коротких глобально согласованных фаз.

Кто выполняет маркировку GC

выделенные работники

частичные работники

свободные работники

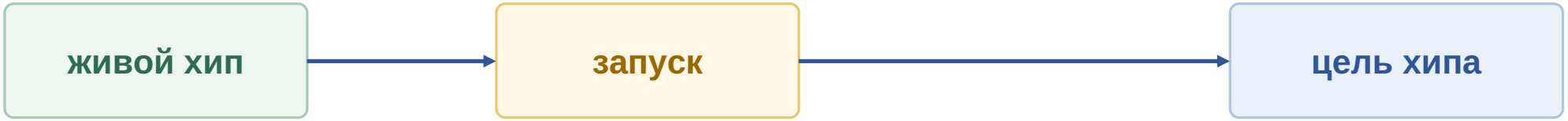
помощь от программы

цель фона ≈ 25% × GOMAXPROCS

Профиль CPU GC не равен потере CPU запросами: работники в простое используют свободные мощности.

25% — ориентир регулятора, не постоянная гарантия доли CPU.

Регулятор GC: цель хипа — финиш, запуск — старт



аллокации продолжаются при маркировке

Начать на границе цели — уже поздно.

Регулятор учитывает аллокации, объем сканирования, скорость, корни и лимит памяти.

GOGC глубже: бюджет роста хипа


```
Цель ≈
Живой хип +
  (Живой хип + корни GC) × GOGC /
100
```

GOGC ↓

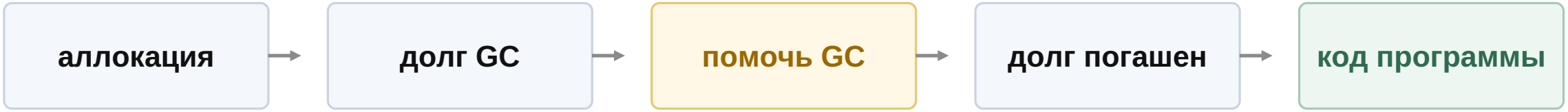
меньше запаса
чаще GC
больше CPU

GOGC ↑

больше запаса
реже GC
больше памяти

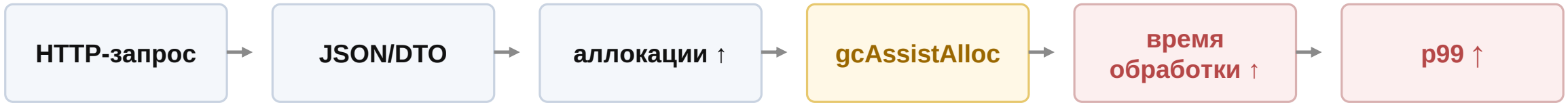
Цель хипа — ориентир. Момент запуска рассчитывается отдельно.

Помощь GC: приложение участвует в уборке памяти



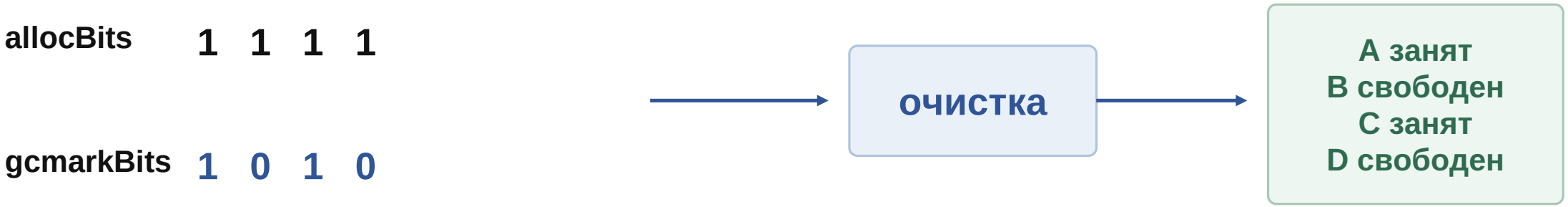
Программа выделяет память быстрее, чем справляется GC → горутина запроса сама выполняет работу сборщика.

Почему помощь GC увеличивает p99



Даже без большого STW задержка растет внутри горутины запроса.

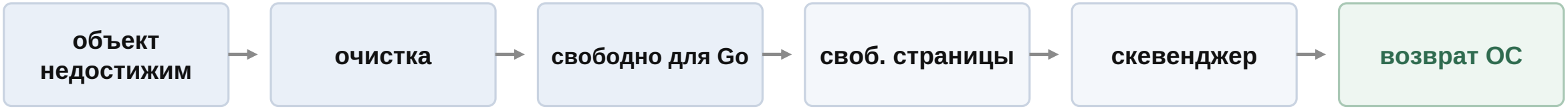
Очистка: маркировка еще не освобождает ячейку



Маркировка: кто жив? Очистка: какие ячейки снова свободны?

Очистка идет в фоне или при аллокации; sweepgen отслеживает состояние спана между циклами.

Очистка ≠ возврат страниц ОС



освобождение для GC ≠ возврат ОС

Рантайм может сохранить свободную память для повторного использования.

Скевенджер: баланс повторного использования и RSS

Отдать все сразу

- RSS снижается быстрее
- но затем снова ошибки страниц и работа ядра
- дороже следующий всплеск нагрузки

Оставить все рантайму

- повторное использование быстрое
- меньше обращений к ядру
- но RSS остается высоким

Скевенджер ищет свободные страницы рантайма, физическую память которых можно вернуть ОС.

Linux: память рантайма и `madvise`



MADV_FREE / MADV_DONTNEED — выбор реализации для платформы

виртуальное адресное пространство ≠ физическая память в RSS

HeapAlloc и RSS: четыре разные величины

живые объекты	свободный хип	возвращено ОС	RSS
700 MB	1.6 GB	900 MB	3.2 GB

HeapAlloc упал → RSS не обязан упасть синхронно.

Сначала память освобождается для Go, затем часть можно вернуть ОС.

GOMEMLIMIT: мягкий бюджет и ограничитель CPU GC

GOMEMLIMIT
900 MiB

живые данные
850 MiB

запас
≈ очень мало

CPU GC
ограничитель

Рантайм может временно превысить мягкий лимит, чтобы не застрять в бесконечном GC.

Мягкий лимит сохраняет прогресс. Жесткий лимит cgroup может вызвать OOM.

Kubernetes: два разных лимита



Нужен запас для памяти вне бюджета GC: cgo, отображения, стеки, расходы платформы.

GOMEMLIMIT = лимит контейнера — плохое правило по умолчанию.

Частые аллокации и удержание памяти: разные инциденты

Сервис A: частые аллокации

```
живой хип = 200 МБ
аллокации = 8 ГБ/с

RSS стабилен
высокая нагрузка GC
```

Сервис B: удержание

```
аллокации = 200 МБ/с
живой хип:
500 → 700 → 1000 → 1500 MB
```

alloc_space

inuse_space

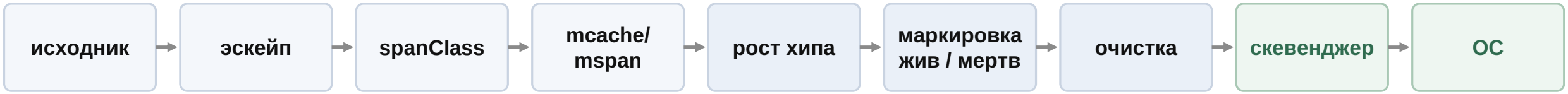
Небольшой живой хип не гарантирует дешевый GC.

Как внутренние механизмы видны в pprof и runtime/metrics

runtime.mallocgc	**runtime.gcAssistAlloc**	**runtime.gcBgMarkWorker**
нагрузка аллокациями	горутина погашает долг GC	фоновая маркировка
/gc/scan/heap	**/gc/heap/goal**	**heap/free/released**
сканируемый хип	цель регулятора GC	аллокатор / возврат ОС

Символ ≠ диагноз. Профиль дает гипотезу, метрики проверяют механизм.

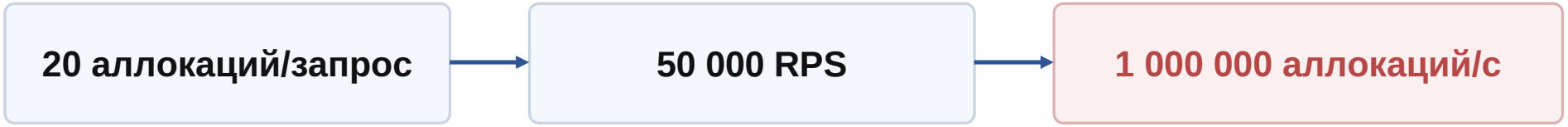
Один объект хипа: полный жизненный цикл

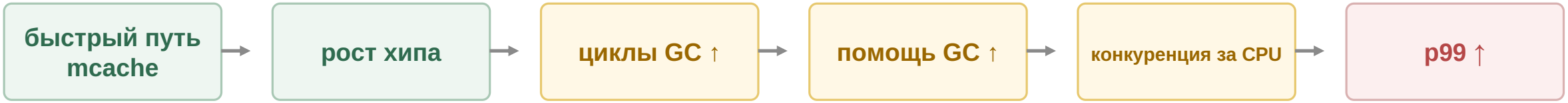


Объект умер → ячейка свободна → страницы свободны → RSS может уменьшиться позже.

Цена аллокации распределена по всему жизненному циклу, не только в момент `new`.

Под нагрузкой небольшая аллокация нагружает весь рантайм





Быстрый путь аллокатора дешев, но сервис позже платит за работу GC.

`sync.Pool`: сокращает аллокации, но имеет цену

Что выигрываем

- повторное использование объектов и буферов
- скорость аллокаций ↓
- работа GC ↓
- меньше временных объектов

Что платим

- удерживаемая память может ↑
- сложнее управление и корректность
- нужно сбрасывать состояние
- не гарантирует сохранность объектов

Pool — после профиля и измерений. Не каждая аллокация требует пула.

Практика: доказать устройство рантайма

классы размеров	scan и noscan	частые аллокации
GOMEMLIMIT	скевенджер	pprof + метрики

сделали → нагрузили → увидели симптом → измерили → объяснили механизм → исправили → снова измерили

Что важно запомнить

- Быстрый путь аллокации — `mcache` + `mspan` на P; общие уровни нужны для пополнения.
- `mspan` связывает аллокатор и GC: ячейки, биты выделения и маркировки находятся вместе.
- Классы размеров ускоряют аллокации ценой фрагментации; scan/noscan определяет цену GC.
- Green Tea улучшает локальность маркировки; это деталь реализации, не гарантия языка.
- Барьер записи — цена корректного обхода меняющегося графа объектов.
- Цель хипа — финиш, порог запуска — старт; регулятор учитывает нагрузку.
- Помощь GC связывает интенсивность аллокаций с задержкой запросов.
- Очистка освобождает ячейки для Go; скевенджер возвращает страницы ОС.
- HeapAlloc, свободный хип, возвращенная память и RSS — разные величины.
- GOMEMLIMIT мягкий, cgroup жесткий. Слишком тесный бюджет памяти нагружает CPU.

Источники для преподавателя

- Аллокатор Go — https://go.dev/src/runtime/malloc.go
- mcache — https://go.dev/src/runtime/mcache.go
- mcentral — https://go.dev/src/runtime/mcentral.go
- mheap / mspan — https://go.dev/src/runtime/mheap.go
- Цикл GC — https://go.dev/src/runtime/mgc.go
- Регулятор GC — https://go.dev/src/runtime/mgcpacer.go
- Барьер записи — https://go.dev/src/runtime/mbarrier.go
- Green Tea — https://go.dev/blog/greenteagc
- Руководство по GC Go — https://go.dev/doc/gc-guide
- runtime/metrics — https://pkg.go.dev/runtime/metrics

Конкретные структуры и константы рантайма относятся к деталям реализации версии Go.